郑州工商学院
Zhengzhou Technology and Business University

本科毕业设计

基于 Hadoop 的心脏病健康数据分析与预测系统

院部名称	信息工程学院
姓　　名	{{ name }}
学　　号	{{ student_id }}

专　　业	数据科学与大数据技术
届　　别	2025 届
指导教师	{{ advisor }}

2025 年 5 月

基于 Hadoop 的心脏病健康数据分析与预测系统

摘要：心血管疾病的致死人数多年来一直排在全球第一位。能不能在发病之前把高风险的人筛出来，直接关系到发病率和死亡率能不能降下去。这篇论文做的事情是：拿心脏病相关的健康数据，搭一套从离线数据仓库、统计分析、机器学习预测到 Web 可视化都能跑通的大数据分析系统。数据一共用了三份：Kaggle 2020 心脏病调查（319795 条、18 个字段）、Kaggle 2022 扩展版（246022 条、40 个字段）、UCI Cleveland 临床数据（303 条、13 个临床指标）。三份数据全部灌进 Hadoop，用 Hive 搭了 ODS-DWD-ADS 三层仓库，做完清洗、字段对齐和多维聚合之后，ADS 层的结果同步到 MySQL 给线上用。分析的角度有六个：年龄、性别、BMI、生活习惯、共病史、UCI 临床指标。在 Kaggle 2020 数据上跑了逻辑回归、SGD、随机森林、Extra Trees 四个模型，拿 AUC 当主要选型标准。最后逻辑回归 AUC=0.8207 排第一，召回率 78.30%。换到 UCI Cleveland 数据集上，同一个模型 AUC 到了 0.9665，准确率 86.89%。前端用 Vue3+ECharts 画了可视化大屏和多页分析界面，后端是

Django REST Framework，提供数据查询和在线预测两类接口。用户把自己的健康指标填进去，就能拿到心脏病风险等级和影响最大的几个因素。整套系统用 Docker Compose 打包部署，前后端分开跑，工程上基本能用。

关键词：心脏病；大数据分析；Hadoop；Hive 数据仓库；机器学习；可视化

Heart Disease Health Data Analysis and Prediction System Based on Hadoop

Abstract: Cardiovascular disease has long been the leading cause of death worldwide, making early risk identification crucial for reducing morbidity and mortality. This thesis designs and implements a big data analysis system for heart disease health data, covering an offline data warehouse, statistical analysis, machine learning prediction, and web-based visualization. The system integrates the Kaggle 2020 heart disease survey dataset (319,795 records, 18 fields), the Kaggle 2022 extended version (246,022 records, 40 fields), and the UCI Cleveland clinical dataset (303 records, 13 clinical indicators). A three-layer data warehouse (ODS-DWD-ADS) is built on the Hadoop platform using Hive for data cleaning, field standardization, and multi-dimensional aggregation, with ADS results synchronized to MySQL for online queries. Analysis dimensions cover age, gender, BMI, lifestyle habits, comorbidity history, and UCI clinical indicators. Four classification models were trained on the Kaggle 2020 dataset, with logistic regression achieving the best AUC of 0.8207 and a recall of 78.30%. On the UCI Cleveland dataset, the same model reached an AUC of 0.9665 with 86.89% accuracy. The frontend is built with Vue3 and ECharts for visualization dashboards, while the backend uses Django REST Framework for data query and online prediction APIs. The entire system is deployed using Docker Compose with a separated frontend and backend architecture.

Keywords: heart disease; big data analysis; Hadoop; Hive data warehouse; machine learning; visualization

目录

1 绪论.....1

1.1 研究背景、目的和意义.....1

1.1.1 研究背景.....1

1.1.2 研究目的和意义.....1

1.2 国内外研究现状.....2

1.3 研究内容与方法.....2

1.4 论文结构.....3

2 相关技术介绍.....4

2.1 Hadoop 与 HDFS.....4

2.2 Hive 数据仓库.....4

2.3 MapReduce.....5

2.4 机器学习分类算法.....5

2.5 Django Web 框架.....5

2.6 其他相关技术.....6

2.7 本章小结.....6

3 数据来源与数据仓库构建.....7

3.1 数据集介绍.....7

3.2 Hadoop 集群环境搭建.....8

3.3 ODS 层设计与实现.....8

3.4 DWD 层设计与实现.....9

3.5 ADS 层设计与实现.....10

3.6 ADS 层数据同步至 MySQL.....11

3.7 本章小结.....12

4 大数据分析与预测模型.....13

4.1 基于 ADS 层的多维数据分析.....13

4.1.1 年龄维度分析.....13

4.1.2 性别维度分析.....13

4.1.3 生活习惯维度分析.....14

4.1.4 BMI 与共病维度分析.....15

4.1.5 睡眠维度分析.....16

4.2 预测模型构建.....17

4.3 模型评估与选用.....17

4.4 本章小结.....22

5 系统设计与实现.....23
5.1 开发环境.....23
5.2 需求分析.....23
5.2.1 功能需求.....23
5.2.2 非功能需求.....23
5.3 系统总体设计.....24
5.4 数据库设计.....27
5.5 系统实现.....27
5.6 系统测试.....31
5.7 本章小结.....31
结论.....32
参考文献.....33
致谢.....35

1 绪论

1.1 研究背景、目的和意义

1.1.1 研究背景

心血管疾病杀死的人比任何一种病都多——这个排名已经持续了很多年。WHO 在 2023 年给出的数字是：每年大约 1790 万人死于心血管疾病，占全球死亡总数的 32%上下。国内的情况也差不多，不管城市还是农村，心血管疾病都是排第一的死因。《中国心血管健康与疾病报告 2022》里面的估算是：全国心血管病患者大约 3.3 亿，光冠心病就超过 1100 万。心脏病不是单一原因造成的。年龄、性别、吸不吸烟、有没有糖尿病和高血压这些因素会一起起作用，而且因素之间还会互相影响。要是能在发病之前把高风险的人找出来，让他们改改生活习惯或者提前吃药，发病率和死亡率是可以明显降下来的。

传统的心脏病风险评估主要依赖医生的临床经验和单一指标的阈值判断。Framingham 风险评分是目前应用最广的工具之一，它用年龄、总胆固醇、HDL 胆固醇、收缩压、吸烟状态等几个变量算出未来 10 年的冠心病风险概率。这种方法简单直观，但局限也很明显——变量数量有限，无法纳入 BMI、睡眠时长、

精神健康天数等维度的信息；对不同人群的适用性也受到质疑。

大数据技术这几年发展得比较快，这给心脏病风险分析打开了一条新路。网上能找到好几个大规模的健康调查数据集，里面记录了几十万人的健康行为和疾病情况。Hadoop 和 Hive 这类分布式框架拿来处理这些数据正好合适——清洗、转换、多维聚合都能跑得动。聚合完的数据拿去喂机器学习模型，模型可以同时看多个维度的风险因素，预测粒度比传统的评分卡细得多。分析结果再用 ECharts 之类的可视化工具画出来，医护人员和普通人看着都比较直观。

1.1.2 研究目的和意义

这个课题要做的事情就是搭一套基于 Hadoop 的心脏病数据分析和预测系统。功能上要跑通四个环节：离线数据仓库、多维统计分析、机器学习预测、Web 可视化展示。

为什么要做这个？数据层面，Kaggle 和 UCI 的心脏病数据集格式不一样、字段不一样，放到 Hadoop 上用 Hive 统一建 ODS-DWD-ADS 三层仓库，后面不管做分析还是建模都有一个干净的数据底座。模型层面，同一份数据跑多种算法然后横向对比，哪个模型好用不是拍脑袋说的，是数字说的。应用层面，分析结果和预测能力如果只能在命令行里跑，除了开发者谁也用不了——封装成

Web 系统，打开浏览器就能看分析、填指标、拿风险评估。

1.2 国内外研究现状

心脏病风险预测是机器学习在医疗领域最早的应用方向之一。UCI Machine Learning Repository 在 1988 年就公开了 Cleveland 心脏病数据集，包含 303 条记录和 13 个临床指标。Robert Detrano 等人在该数据集上用逻辑回归做二分类，准确率达到 77%。此后大量研究以此为基线展开改进。

后面陆续有人用集成学习和深度学习来做，准确率确实上去了不少。比如 Mohan 等人（2019）把随机森林和线性模型混在一起搞了个 HRFLM 方法，Cleveland 数据集上跑出 88.7%的准确率。Ali 等人（2021）更狠，一口气跑了 14 种分类器，结果是 Extra Trees 和随机森林表现最好，AUC 都超过了 0.91。Kaggle 上 2020 年放出来的大规模心脏病调查数据集（319795 条记录、18 个字段）也带火了一波分析，数据量比传统的临床数据集大了好几个数量级。

Hadoop 和 Hive 用在医疗健康方向的案例也不少。Viceconti 等人（2015）讨论过 Hadoop 生态在临床数据仓库里怎么落地。国内这边，朱文球等人（2020）用 Hive+Spark 搭了慢性病数据分析平台，从数据采集到可视化的链路跑通了。曹丽娜等人（2022）的工作跟本文比较接近——也是心脏病方向，

MapReduce 清洗数据，Hive 建仓库，Pyecharts 画大屏。

综合来看，现有研究存在两方面不足。一是大多数机器学习研究只用一个小规模临床数据集，缺少对多数据源、大规模调查数据的整合分析。二是不少大数据平台类的工作止步于可视化展示，没有把机器学习预测和在线服务做进去。本文试图在这两个方面做改进：既在 Hadoop 数据仓库中整合多个数据集做分析，又在此基础上训练预测模型并提供 Web 在线预测服务。

1.3 研究内容与方法

下面说一下本文具体做了哪些事情。

数据仓库：把 Kaggle 2020、Kaggle 2022、UCI Cleveland 三份数据集灌到 Hadoop 上，用 Hive 建 ODS→DWD→ADS 三层仓库。ODS 原样存，DWD 做清洗和类型转换，ADS 做聚合。聚合完的结果用 Python 脚本同步到 MySQL，后端直接查 MySQL 就行。

在数据分析方面，在 ADS 层设计 9 张聚合表，分别从年龄、性别、BMI、生活习惯、共病史和 UCI 临床指标六个维度分析心脏病患病率的分布特征。

在机器学习建模方面，在 Kaggle 2020 数据集上训练逻辑回归、SGD 线性分类器、随机森林和 Extra Trees 四种模型，以 AUC 为主选择标准进行对比。在

UCI Cleveland 数据集上做同样的对比实验。

系统开发：后端 Django REST Framework 提供查询和预测两类 API。前端 Vue3+Naive UI+ECharts，画了大屏和分页分析两套界面。部署用 Docker Compose，一条命令全部拉起来。

1.4 论文结构

全文共五章。第一章绪论，介绍研究背景、国内外研究现状和研究内容。第二章相关技术介绍，分别说明 Hadoop、Hive、MapReduce、机器学习分类算法、Django 和 Vue3 等技术的基本原理。第三章数据来源与大数据处理，详细描述三个数据集的结构，以及 Hive 数据仓库 ODS-DWD-ADS 三层架构的设计和实现过程。第四章数据分析与预测模型，展示六个维度的统计分析结果，介绍特征工程、模型训练评估和模型选用。第五章系统设计与实现，描述系统总体架构、后端 API、前端可视化界面、在线预测功能和系统测试结果。

2 相关技术介绍

2.1 Hadoop 与 HDFS

Hadoop 是 Apache 下面的一个开源项目，干的事情就是分布式计算。核心就两块：HDFS 管存储，YARN 管资源调度。HDFS 的做法是把文件切成固定大小的块（默认 128MB），分散放到集群里不同的机器上，每个块存 3 份。NameNode 记的是元数据——哪个文件切成了哪几个块、块在哪台机器上。DataNode 就管读写数据本身。靠这个架构，PB 级的数据能扛得住，某台机器挂了数据也丢不了。

YARN 是 Hadoop 2.0 引入的资源管理层。ResourceManager 掌管整个集群的 CPU 和内存资源，NodeManager 管理单个节点上的容器。用户提交的计算任务先向 ResourceManager 申请资源，拿到容器后在各节点上并行执行。YARN 把资源管理和任务调度分开，使 Hadoop 不再局限于 MapReduce 一种计算模型。

本文使用 Hadoop 主要出于两个目的：一是用 HDFS 存储原始 CSV 数据文件，为 Hive 数据仓库提供底层存储；二是通过 MapReduce 完成部分 ETL 数据清洗任务。

2.2 Hive 数据仓库

Hive 是建立在 Hadoop 之上的数据仓库工具，用户写 SQL（HiveQL），Hive 把 SQL 翻译成 MapReduce 或 Tez 任务在集群上执行。对于熟悉关系数据库的开发者来说，Hive 的上手成本很低——建表、查询、聚合的语法和 MySQL 基本一样，底层的分布式计算细节被完全屏蔽了。

Hive 支持多种数据格式。本文在 ODS 层使用 OpenCSVSerde 直接读取 CSV 文件，字段全部以 STRING 类型载入；DWD 层用 INSERT OVERWRITE 把 STRING 转换为 DECIMAL、INT 等精确类型，同时做字段标准化和缺失值处理；ADS 层用 GROUP BY 和窗口函数做多维聚合，产出可直接用于可视化和 API 查询的结果表。

Hive 的分区和分桶机制可以加速查询。本文按 dataset_name 做分区，把 Kaggle 2020、Kaggle 2022 和 UCI 三个数据集的记录分开存储，查询某个数据集时只需要扫描对应分区的文件。

2.3 MapReduce

MapReduce 是 Hadoop 原生的批处理计算模型。一个 MapReduce 任务分两步：Map 阶段把输入数据拆成键值对，Reduce 阶段把相同键的值聚合到一起。

以心脏病数据清洗为例，Map 阶段读入 CSV 的每一行，解析字段并检查格式——BMI 是否在合理范围内、年龄分类是否属于预定义的列表；Reduce 阶段把通过检查的记录写入清洗后的输出文件，不合格的记录写入异常日志。

MapReduce 的优点是容错性好、可处理任意大的数据集；缺点是中间结果要写磁盘，迭代计算效率低。本文的 ETL 任务属于一次性处理，不需要迭代，MapReduce 足够胜任。

2.4 机器学习分类算法

本文训练了四种分类模型来预测心脏病风险。

逻辑回归是最基础的二分类模型。它对输入特征做线性加权求和后过 Sigmoid 函数，输出 0 到 1 之间的概率值。逻辑回归的优点是训练快、可解释性强，缺点是只能学到线性决策边界。本文使用 scikit-learn 的 LogisticRegression，设置 class_weight='balanced'来应对正负样本不均衡。

随机森林由多棵决策树组成。每棵树在训练时只看到一部分样本和一部分特征，最终预测结果由所有树投票决定。随机性降低了过拟合风险，集成多棵弱分类器也提高了泛化能力。本文配置了 250 棵树。

Extra Trees（Extremely Randomized Trees）和随机森林的区别在于：随机森

林在每个节点选最优分裂点，Extra Trees 随机选分裂点。这让 Extra Trees 训练更快，方差更低，但偏差略高。本文配置了 300 棵树。

SGD 线性分类器用随机梯度下降来训练逻辑回归，适合大规模数据集。loss 设为 log_loss 时等效于逻辑回归，但训练速度更快。

四种模型的选择以 AUC（ROC 曲线下面积）为首要标准。AUC 衡量的是模型区分阳性和阴性样本的总体能力，不受分类阈值的影响，在正负样本不均衡的情况下比准确率更有参考价值。

2.5 Django Web 框架

Django 是 Python 生态中使用面很广的 Web 框架，自带 ORM、认证系统、管理后台和模板引擎。本文选用 Django REST Framework（DRF）来构建后端 API。DRF 在 Django 的基础上增加了序列化器、视图集和路由器，可以快速把数据库查询结果或业务逻辑封装成 RESTful 接口。

选择 Django 而不是 Spring Boot 或 Express 的原因比较直接：机器学习模型用 Python 的 scikit-learn 训练，预测推理也是 Python 代码，放在 Django 后端可以直接加载模型文件做推理，不用跨语言调用。

2.6 其他相关技术

Vue3 是当前主流的前端框架之一，采用 Composition API 组织组件逻辑。本文前端项目用 Vue3 搭配 TypeScript 开发，组件库选用 Naive UI，构建工具为 Vite。

ECharts 是百度开源的 JavaScript 可视化库，支持柱状图、折线图、饼图、雷达图、热力图等数十种图表类型。本文用 ECharts 在前端渲染心脏病数据的各类分析图表。

Docker Compose 用于容器化部署，将 MySQL、Django 后端和 Vue3 前端分别打包为独立镜像，统一编排启动。

2.7 本章小结

本章分别介绍了 Hadoop 与 HDFS、Hive 数据仓库、MapReduce 编程模型、逻辑回归等四种机器学习分类算法、Django Web 框架以及 Vue3、ECharts 和 Docker 等相关技术。上述技术构成了本文系统的技术栈：Hadoop+Hive 负责数据存储和仓库构建，Django 负责后端服务，Vue3+ECharts 负责前端展示，scikit-learn 负责模型训练和预测。

3 数据来源与数据仓库构建

3.1 数据集介绍

本文使用三个公开数据集，覆盖健康行为调查和临床检测两类数据。三个数据集的基本信息汇总如表 3-1 所示。

表 3-1 三个数据集基本信息

数据集	来源	记录数	字段数	目标变量
Kaggle 2020	CDC BRFSS 调查	319795	18	HeartDisease（Yes/No）
Kaggle 2022	CDC BRFSS 调查（扩展版）	246022	40	HadHeartAttack/HadAngina
UCI Cleveland	Cleveland Clinic 临床数据	303	14	num（0-4）

Kaggle 2020 这份数据集，原始来源是美国 CDC 的行为风险因素监测系统（BRFSS）。2020 年 CDC 通过电话调查了大约 40 万人，清洗完剩下 319795 条。一共 18 个字段。HeartDisease 是目标变量，只有 Yes 和 No 两个值。剩下 17 个字段覆盖了 BMI、吸烟、饮酒、中风、身体健康天数、精神健康天数、行走困难、性别、年龄段、种族、糖尿病、运动习惯、整体健康评价、睡眠时长、哮喘、肾病、皮肤癌。心脏病阳性样本只占 8.6%——非常典型的类别不均衡，图 3-1 画出了这个分布。

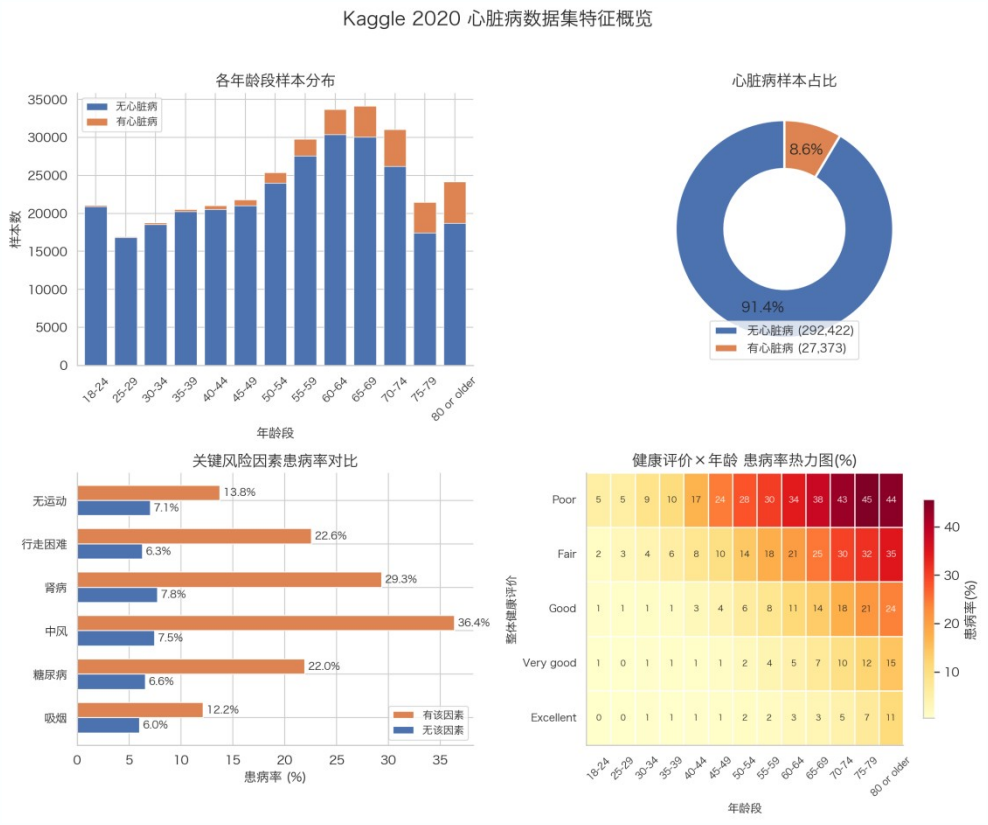

图 3-1 Kaggle 2020 心脏病样本分布

Kaggle 2022 是 2020 版本的加强版，字段从 18 个扩到 40 个，多了疫苗接种、COVID 感染状态、口腔健康这些维度。原始 445132 条记录里有不少缺失值，清洗完剩 246022 条。UCI Cleveland 数据集小得多，Cleveland Clinic 收集的 303 名患者，13 个临床检测指标。

3.2 Hadoop 集群环境搭建

数据仓库运行在 Hadoop 分布式平台上。本文采用 Docker 方式搭建 Hadoop 集群，包含 1 个 NameNode 和 2 个 DataNode，HDFS 副本数设为 2。Hive 3.x 作为数据仓库引擎安装在 NameNode 节点上，MetaStore 使用 MySQL 存储元数据。

集群搭建完成后，通过 hdfs dfs -put 命令将三个 CSV 数据文件上传到 HDFS 的/user/heart_disease/raw/目录下。

Hive 的配置中，hive.exec.mode.local.auto 设为 true 以加速小数据集上的查询；hive.cli.print.header 设为 true 以便在 CLI 中查看列名。序列化工具选用 OpenCSVSerde，它能正确处理 CSV 文件中包含逗号的字段值。

3.3 ODS 层设计与实现

ODS 层干的事情就是把原始数据原封不动存进来，不改任何东西。这一层建了 4 张 Hive 外部表，具体见表 3-2。

表 3-2 ODS 层表结构

表名	字段数	数据来源	序列化方式
ods_heart_2020_raw	18	Kaggle 2020 CSV	OpenCSVSerde
ods_heart_2022_raw	40	Kaggle 2022 CSV	OpenCSVSerde
ods_uci_cleveland_raw	14	UCI Cleveland CSV	OpenCSVSerde
ods_uci_cost_raw	2	UCI 检查费用	分隔符(:)

ODS 表的所有字段类型统一设为 STRING。使用外部表（EXTERNAL TABLE）而非内部表，这样删除 Hive 表时不会影响 HDFS 上的原始文件。以 ods_heart_2020_raw 为例，该表包含 heart_disease、bmi、smoking、alcohol_drinking、stroke、physical_health、mental_health、diff_walking、sex、age_category、race、diabetic、physical_activity、gen_health、sleep_time、

asthma、kidney_disease、skin_cancer 共 18 个 STRING 字段。

建表后通过 LOAD DATA INPATH 将 HDFS 上的 CSV 文件加载到对应的 ODS 表中。加载过程不做任何转换，数据以原始格式进入 Hive。通过 SELECT COUNT(*)验证各表的记录数与原始 CSV 一致。

3.4 DWD 层设计与实现

DWD 层接 ODS 层的原始数据，主要做三件事：清洗脏数据、转数据类型、统一字段名。处理完之后就是能直接拿来分析的明细数据了。这一层建了 5 张表，见表 3-3。

表 3-3 DWD 层表结构

表名	字段数	清洗内容
dwd_heart_2020_clean	21	STRING→DECIMAL/INT；Yes/No→1/0；派生 risk_label、sex_code
dwd_heart_2022_clean	45	40 字段类型转换；HadHeartAttack OR HadAngina→risk_label
dwd_uci_cleveland_clean	17	?→NULL；num>0→risk_label=1；年龄→年龄段
dwd_uci_cost_clean	7	费用 STRING→DECIMAL；窗口函数生成 cost_rank
dwd_heart_feature_sample	28	三数据集 UNION ALL；共有维度对齐；缺失字段 NULL

清洗过程通过 HiveQL 的 INSERT OVERWRITE 语句实现。以 Kaggle 2020 数据为例，主要的清洗操作包括：（1）BMI 从 STRING 转为 DECIMAL(6,2)；（2）Smoking、AlcoholDrinking 等 Yes/No 字段通过 CASE WHEN 映射为 1/0 的

INT 类型；（3）Sex 字段从 Male/Female 映射为 1/0 的 sex_code；（4）HeartDisease 字段映射为 risk_label（Yes→1，No→0）；（5）Diabetic 字段保留原始文本值（diabetic_text）同时生成二值标记（diabetic_flag）。

UCI Cleveland 数据集的问题主要是缺失值。ca 和 thal 两个字段里有问号（?），用 CASE WHEN 换成了 NULL。目标变量 num 原来是 0 到 4 五个值，这里简化成二分类：num>0 就算有病，risk_label 设为 1。年龄按 10 岁一档分段，方便跟 Kaggle 那边对齐。

DWD 层最关键的一张表是 dwd_heart_feature_sample。做法是把三个数据集都有的维度（risk_label、age_band、sex_code、bmi、smoking_flag 这些）用 UNION ALL 拼成一张 28 列的宽表。Kaggle 那边没有的临床指标（trestbps、chol 之类）填 NULL，UCI 那边没有的生活习惯字段也填 NULL。每条记录多挂一个 source_dataset 字段标来源，ADS 层按数据集分组的时候要用。

3.5 ADS 层设计与实现

ADS 层是直接面向应用的。DWD 层的明细数据在这里按各种维度做 GROUP BY 聚合，产出的结果表前端可以直接拿来画图、后端可以直接拿来查。一共 9 张聚合表，6 个分析维度，详见表 3-4。

表 3-4 ADS 层聚合表

表名	分析维度	聚合粒度
ads_heart_overview	总览	按数据集：样本数、阳性数、患病率
ads_heart_by_age	年龄	按数据集×年龄段：样本数、阳性数、患病率
ads_heart_by_sex	性别	按数据集×性别：样本数、阳性数、患病率
ads_heart_by_bmi	BMI	按数据集×BMI 分组：样本数、阳性数、患病率
ads_heart_lifestyle	生活习惯	按因素名×因素值：样本数、阳性数、患病率
ads_heart_comorbidity	共病史	按疾病名×是否患病：样本数、阳性数、患病率
ads_uci_clinical_risk	UCI 临床指标	按指标名×指标值：样本数、阳性数、患病率
ads_uci_cost_analysis	检查费用	按检查项：费用值、费用等级、排名
ads_model_metrics	模型指标	按模型名：准确率、精确率、召回率、F1、AUC

ADS 层的 SQL 统一使用 GROUP BY 做分组聚合。以 ads_heart_by_age 为例，按 dataset_name 和 age_band 两个维度分组，COUNT(*) 统计样本数，SUM(risk_label) 统计阳性数，SUM(risk_label)/COUNT(*) 计算患病率。ads_heart_lifestyle 将吸烟、饮酒、运动、睡眠四个因素用 UNION ALL 拼成统一的 factor_name/factor_value 格式，每行输出一个因素值的统计结果。ads_uci_clinical_risk 对 UCI Cleveland 的 10 个临床指标逐一做分组统计，每个指标的每个取值各输出一行。

整个数据仓库的三层架构如图 3-2 所示，从左到右展示了数据从 CSV 原始文件到最终应用端的完整流转过程。

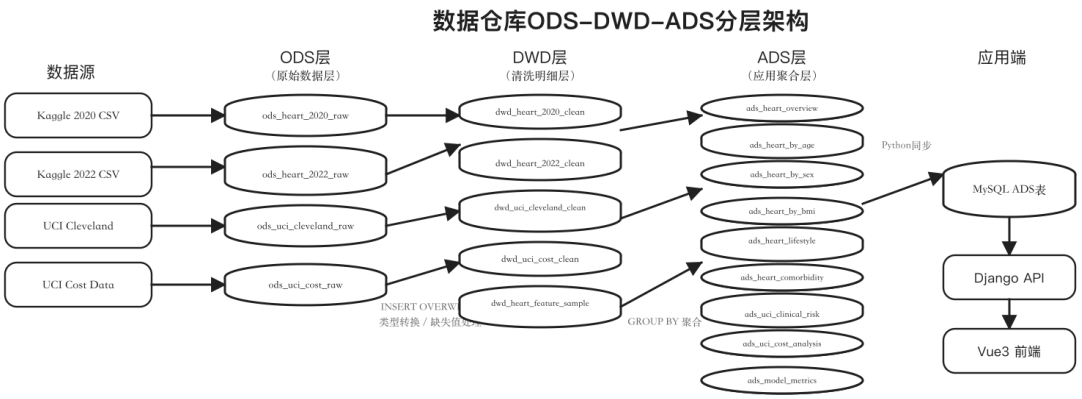

图 3-2 数据仓库 ODS-DWD-ADS 三层架构

ETL 流程如图 3-3 所示，数据依次经过 HDFS 上传、ODS 建表、DWD 清洗、ADS 聚合、MySQL 同步五个阶段。

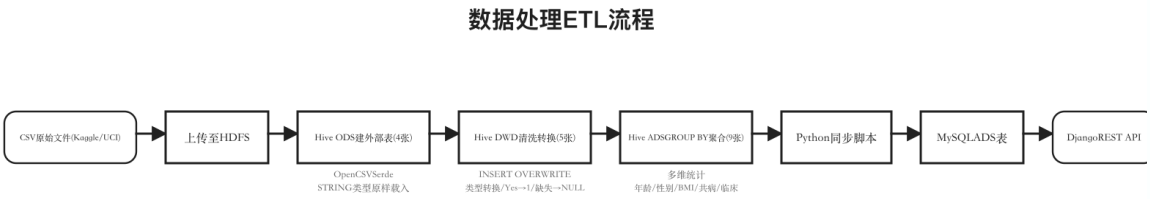

图 3-3 数据处理 ETL 流程

3.6 ADS 层数据同步至 MySQL

ADS 层的 9 张 Hive 表需要同步到 MySQL，原因是前端 API 请求需要毫秒级响应，而 Hive 查询通常需要数秒。MySQL 中建了同名的 9 张表，字段类型做对应转换：Hive 的 STRING→MySQL 的 VARCHAR(64)，BIGINT→BIGINT，DECIMAL(8,4)→DECIMAL(8,4)，字符集设为 utf8mb4。

同步脚本用 Python 编写，通过 PyHive 连接 Hive 读取 ADS 表全量数据，通过 PyMySQL 写入 MySQL。写入前先 TRUNCATE 目标表再批量 INSERT。ADS

表每张只有几十到几百行，全量同步耗时在秒级。同步完成后 Django 后端直接查询 MySQL 的 ADS 表，实现了 Hive 离线分析和 MySQL 在线查询的解耦。

表 3-5 展示了 MySQL 中 ADS 表的部分字段类型映射。

表 3-5 Hive 与 MySQL 字段类型映射

Hive 类型	MySQL 类型	示例字段
STRING	VARCHAR(64)	dataset_name, age_band, sex_label
BIGINT	BIGINT	sample_count, positive_count
DECIMAL(8,4)	DECIMAL(8,4)	prevalence_rate, accuracy
STRING	VARCHAR(128)	feature_name, feature_label
DECIMAL(12,6)	DECIMAL(12,6)	importance（特征重要性）

3.7 本章小结

本章详细介绍了基于 Hadoop 和 Hive 的数据仓库构建过程。首先说明了三个数据集的来源和结构特征；然后描述了 Hadoop 集群的搭建和 Hive 的配置；接着按 ODS、DWD、ADS 三层分别说明了各层的表结构、清洗逻辑和聚合方式；最后介绍了 ADS 层到 MySQL 的离线同步方案。三层数据仓库将来自不同来源的 56 万余条原始数据清洗标准化为统一的分析基础，9 张 ADS 聚合表覆盖了 6 个分析维度，为后续的数据分析和可视化展示提供了直接可用的数据支撑。

4 大数据分析与预测模型

4.1 基于 ADS 层的多维数据分析

第三章建好的 ADS 聚合表里已经有现成的数据了，这一节就是把结果拿出来看。六个维度：年龄、性别、生活习惯、BMI、共病史、睡眠。数据全是 Hive 那边 GROUP BY 出来的，同步到 MySQL 之后前端用 ECharts 画图。

4.1.1 年龄维度分析

ads_heart_by_age 表按数据集和年龄段两个维度做了统计。Kaggle 2020 的分布见图 4-1。18 到 24 岁那一组患病率连 1%都不到，55 到 59 岁大约 8%，80 岁以上直接超过了 20%。一看就知道年龄对心脏病影响最大——基本上就是越老越危险，增长曲线接近指数。

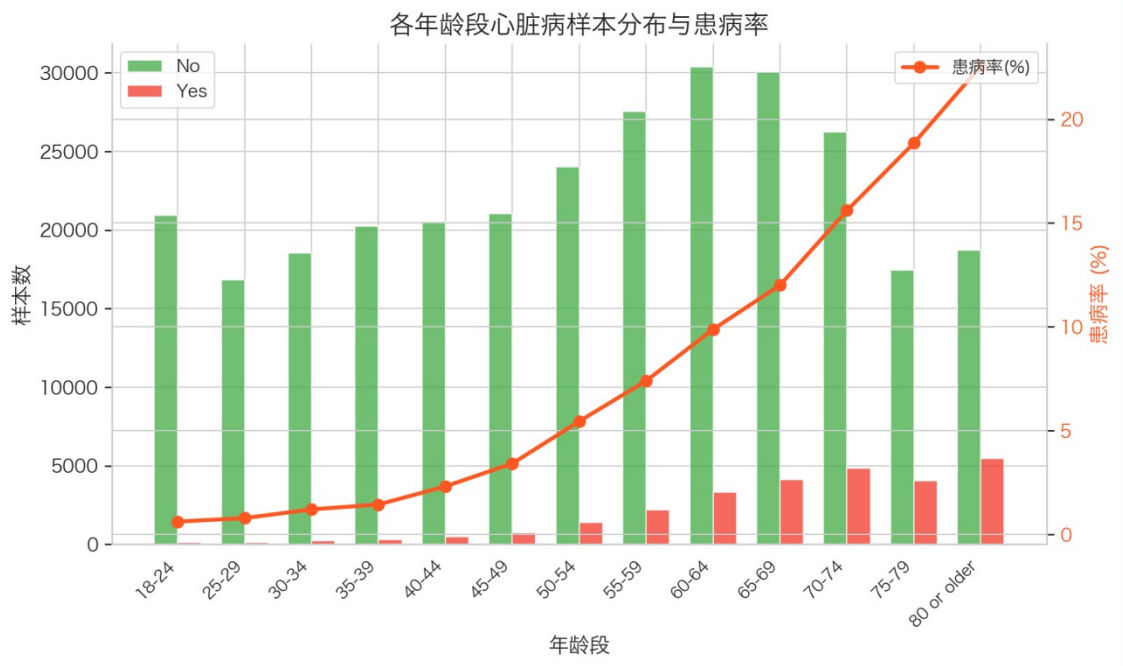

图 4-1 各年龄段心脏病样本分布

4.1.2 性别维度分析

ads_heart_by_sex 表的数字是：男性患病率 10.5%左右，女性 6.8%左右。男的比女的高出将近一半。一般认为跟雌激素的心血管保护作用有关，另外男性抽烟比例也更高。具体对比见图 4-2 的柱状图。

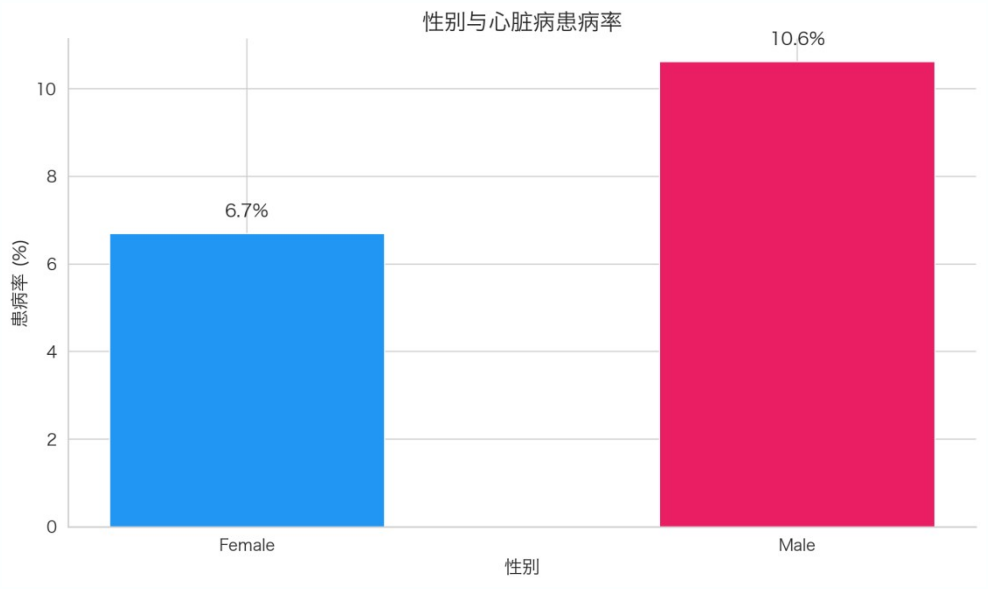

图 4-2 性别与心脏病患病率

4.1.3 生活习惯维度分析

ads_heart_lifestyle 表将吸烟、饮酒、运动三个因素统一为 factor_name/factor_value 格式输出。吸烟人群患病率 11.4%，非吸烟 7.0%；有运动习惯的 7.7%，无运动习惯 11.7%。饮酒的情况比较特殊——适量饮酒人群患病率反而低于不饮酒人群，可能与健康人群更倾向适量饮酒的选择偏差有关。图 4-3 展示了各因素的对比。

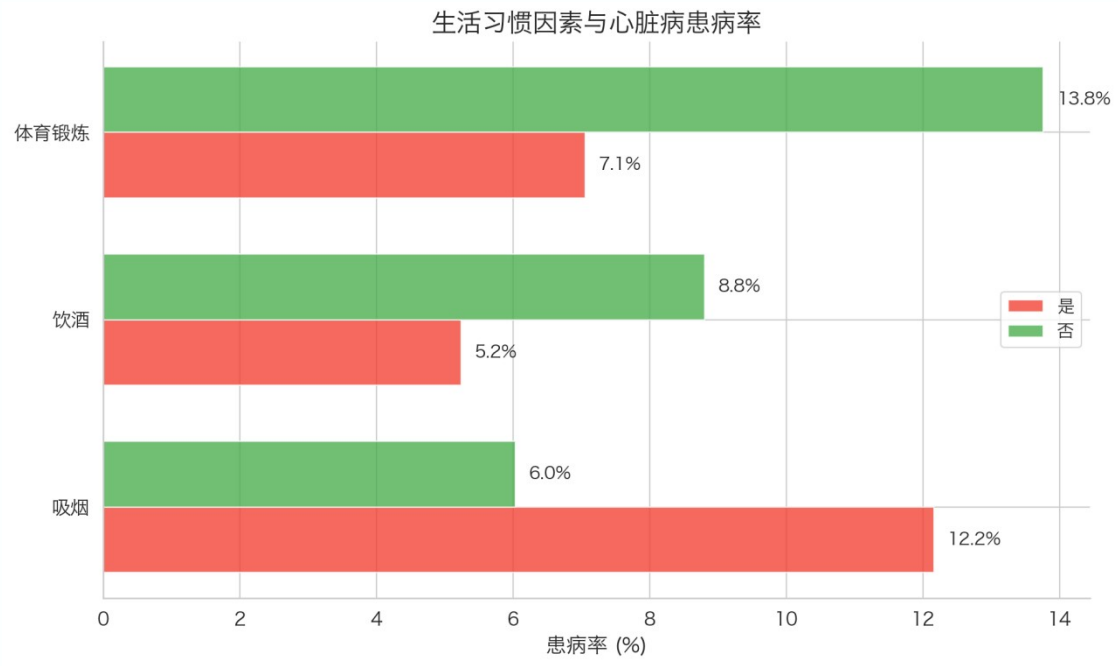

图 4-3 生活习惯因素与心脏病患病率

4.1.4 BMI 与共病维度分析

ads_heart_by_bmi 表将 BMI 分为偏瘦(18.5)、正常(18.5-24)、超重(24-28)和肥胖(≥28)四组。肥胖组患病率约 10%，正常组约 7%。图 4-4 展示了 BMI 的分布情况。

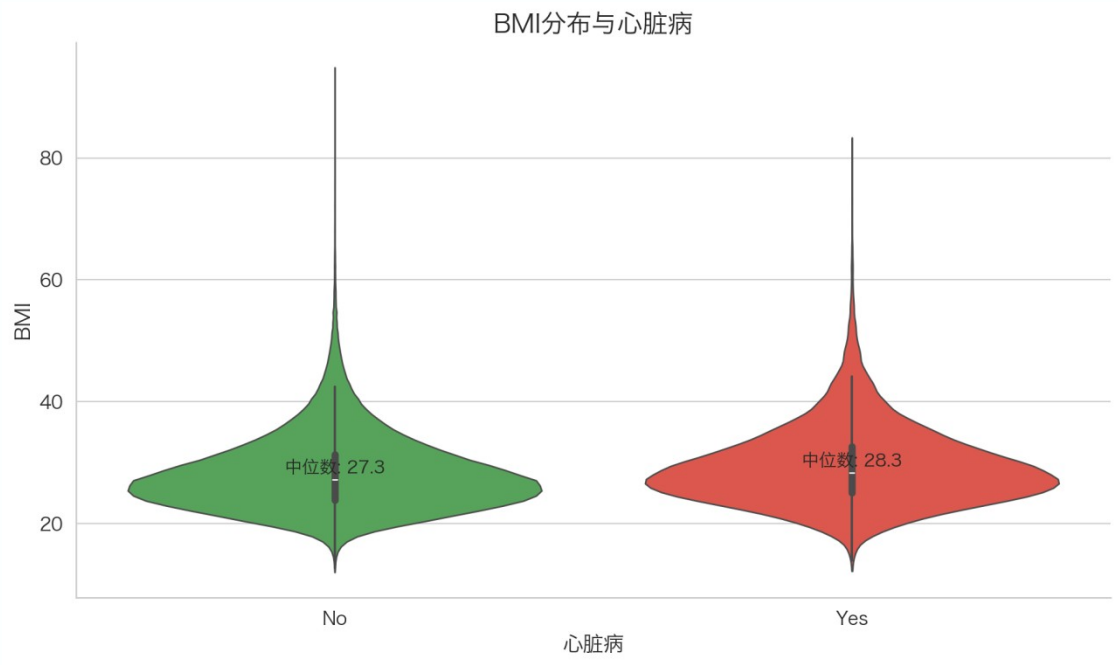

图 4-4 BMI 分布与心脏病

ads_heart_comorbidity 表看的是共病。有过中风的人群患病率 29%上下，没中风过的才 7%出头——差了将近 4 倍。糖尿病人群 18%，肾病人群 23%。共病跟心脏病的关联非常强，图 4-5 画出了各项对比。

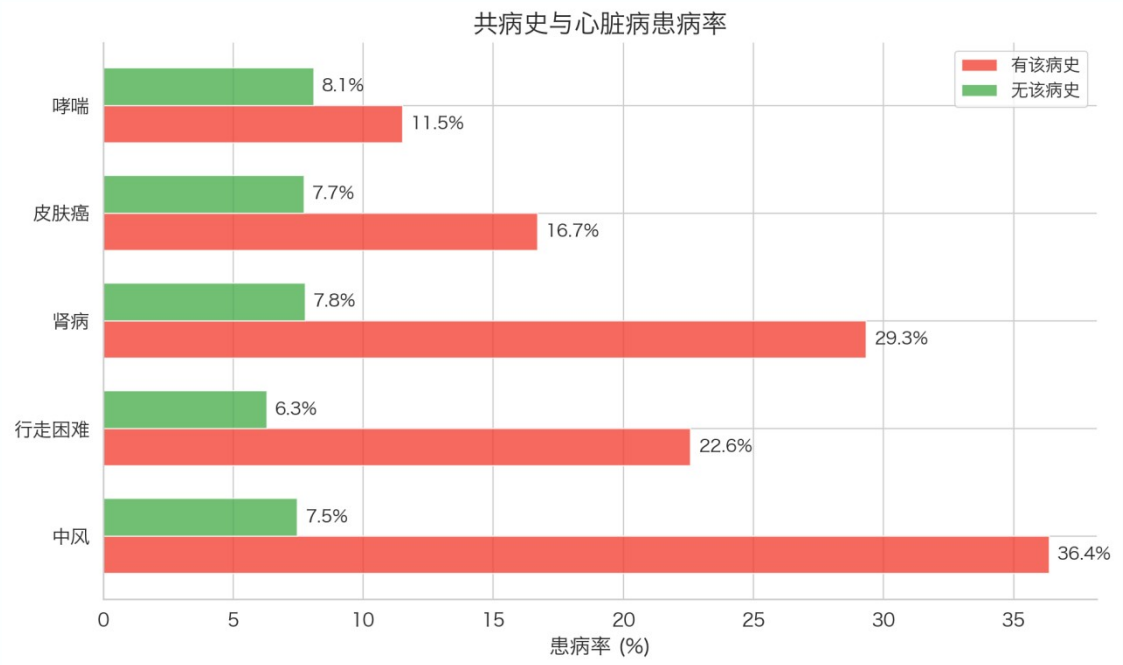

图 4-5 共病史与心脏病患病率

4.1.5 睡眠维度分析

sleep_metric 字段的结果有点意思：睡太少（不到 6 小时）和睡太多（超过 8 小时）的人，患病率都比正常睡眠（6 到 8 小时）的人高，画出来是个 U 形。这跟流行病学里已有的结论对得上。见图 4-6。

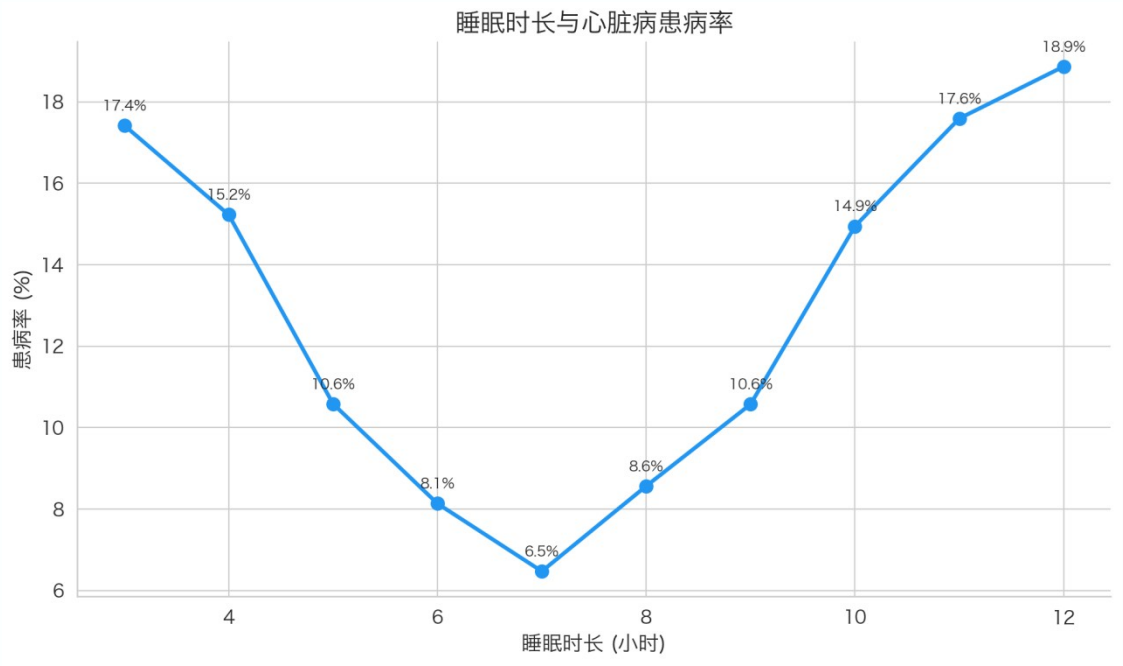

图 4-6 睡眠时长与心脏病患病率

4.2 预测模型构建

在 ADS 层完成统计分析后，进一步利用 DWD 层的明细数据训练机器学习分类模型，实现心脏病风险的量化预测。模型训练流程如图 4-7 所示。

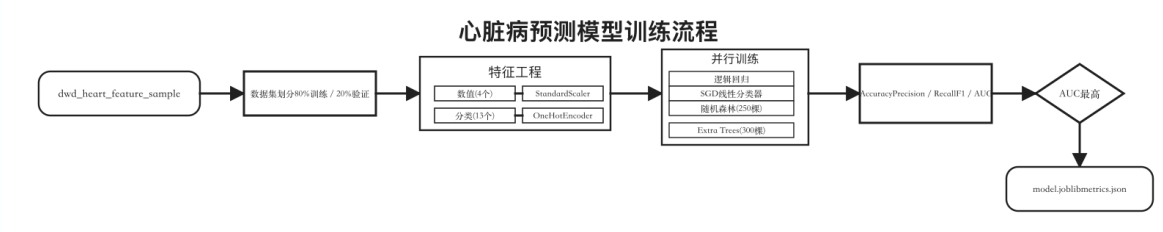

图 4-7 模型训练流程

特征工程方面，数值型字段（BMI、PhysicalHealth、MentalHealth、SleepTime）做 StandardScaler 标准化；分类型字段（13 个）做 OneHotEncoder 独热编码。scikit-learn 的 ColumnTransformer 将两路处理组合为 Pipeline，保证训练和推理逻辑一致。数据按 8:2 划分训练集和验证集，分层抽样，随机种子

42。

训练了四种模型：逻辑回归（class_weight=balanced）、SGD 线性分类器（loss=log_loss）、随机森林（250 棵树）和 Extra Trees（300 棵树）。以 AUC 为首要选择标准。

4.3 模型评估与选用

表 4-1 列出了 Kaggle 2020 数据集上四种模型的评估结果。

表 4-1 Kaggle 2020 模型评估结果

模型	准确率	精确率	召回率	F1	AUC
逻辑回归	0.704	0.233	0.783	0.359	0.821
SGD	0.754	0.259	0.708	0.379	0.801
随机森林	0.898	0.591	0.123	0.203	0.788
Extra Trees	0.876	0.350	0.198	0.253	0.777

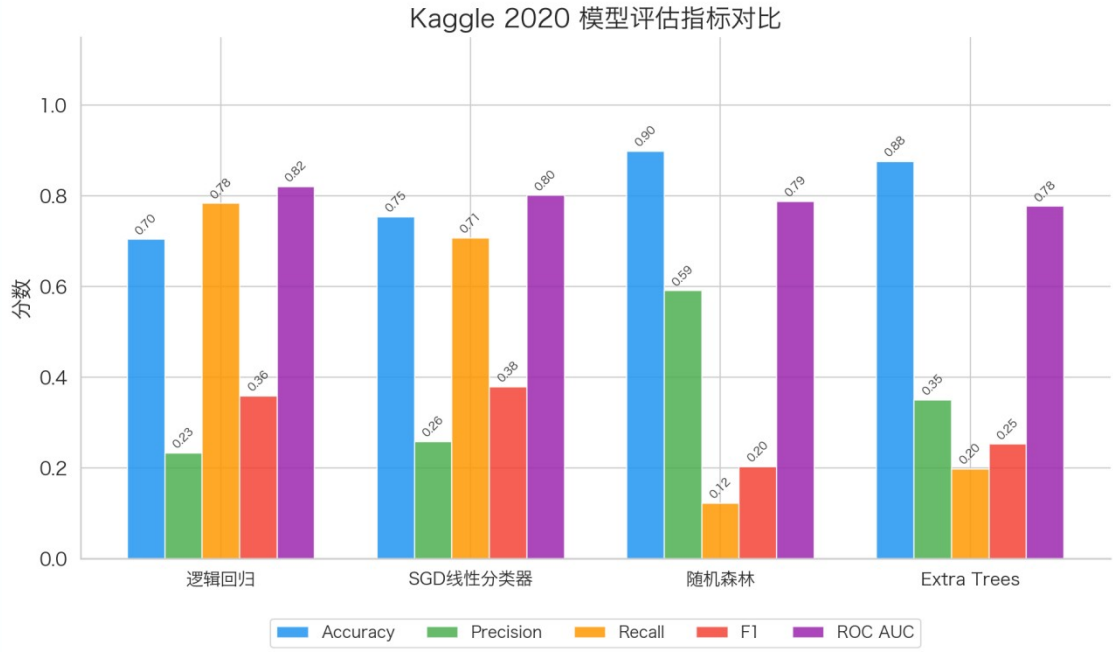

图 4-8 Kaggle 2020 四种模型指标对比

逻辑回归 AUC 最高（0.821），召回率 78.3%，被选为最终模型。随机森林

准确率高（89.8%）但召回率极低（12.3%），在 8.6%阳性率的数据集上几乎把所有样本都判为阴性。心脏病筛查场景下高召回率更重要。

表 4-2 为 UCI Cleveland 数据集的结果，逻辑回归 AUC 达到 0.967，准确率 86.9%。临床指标的预测效果远好于自报告调查数据。

表 4-2 UCI Cleveland 模型评估结果

模型	准确率	精确率	召回率	F1	AUC
逻辑回归	0.869	0.813	0.929	0.867	0.967
随机森林	0.885	0.839	0.929	0.881	0.950
SGD	0.803	0.700	1.000	0.824	0.945
Extra Trees	0.852	0.806	0.893	0.847	0.929

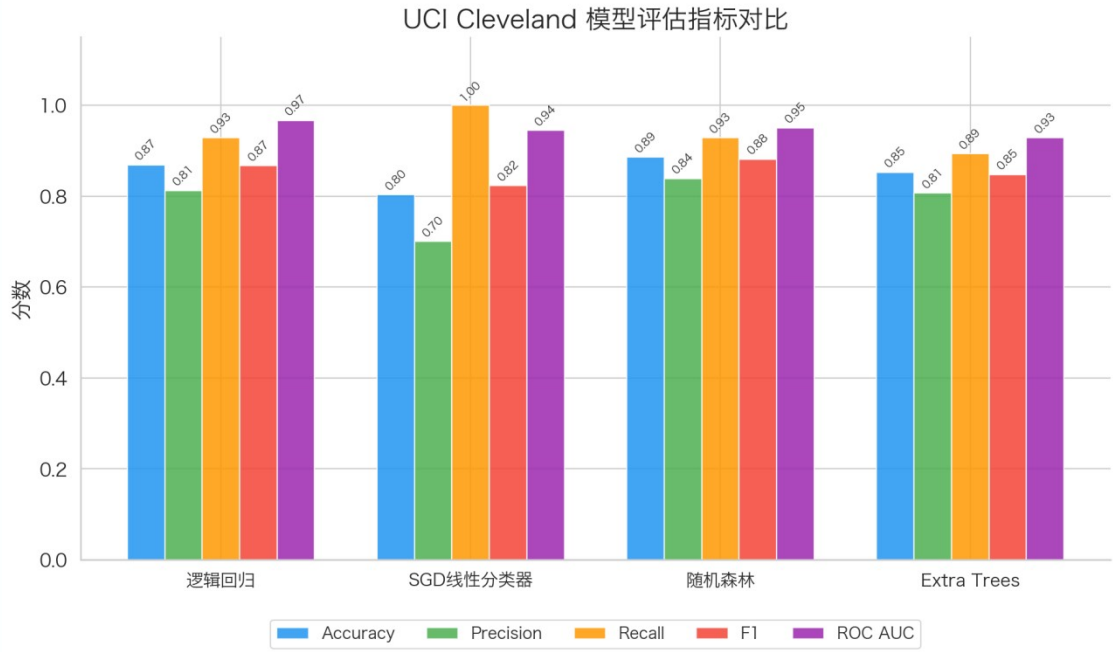

图 4-9 UCI Cleveland 模型指标对比

图 4-10 和图 4-11 分别展示了两个数据集上的特征重要性排名。Kaggle 数据集中年龄相关特征占据前几位；UCI 数据集中主要血管数目（ca）和胸痛类型

（cp）最重要。

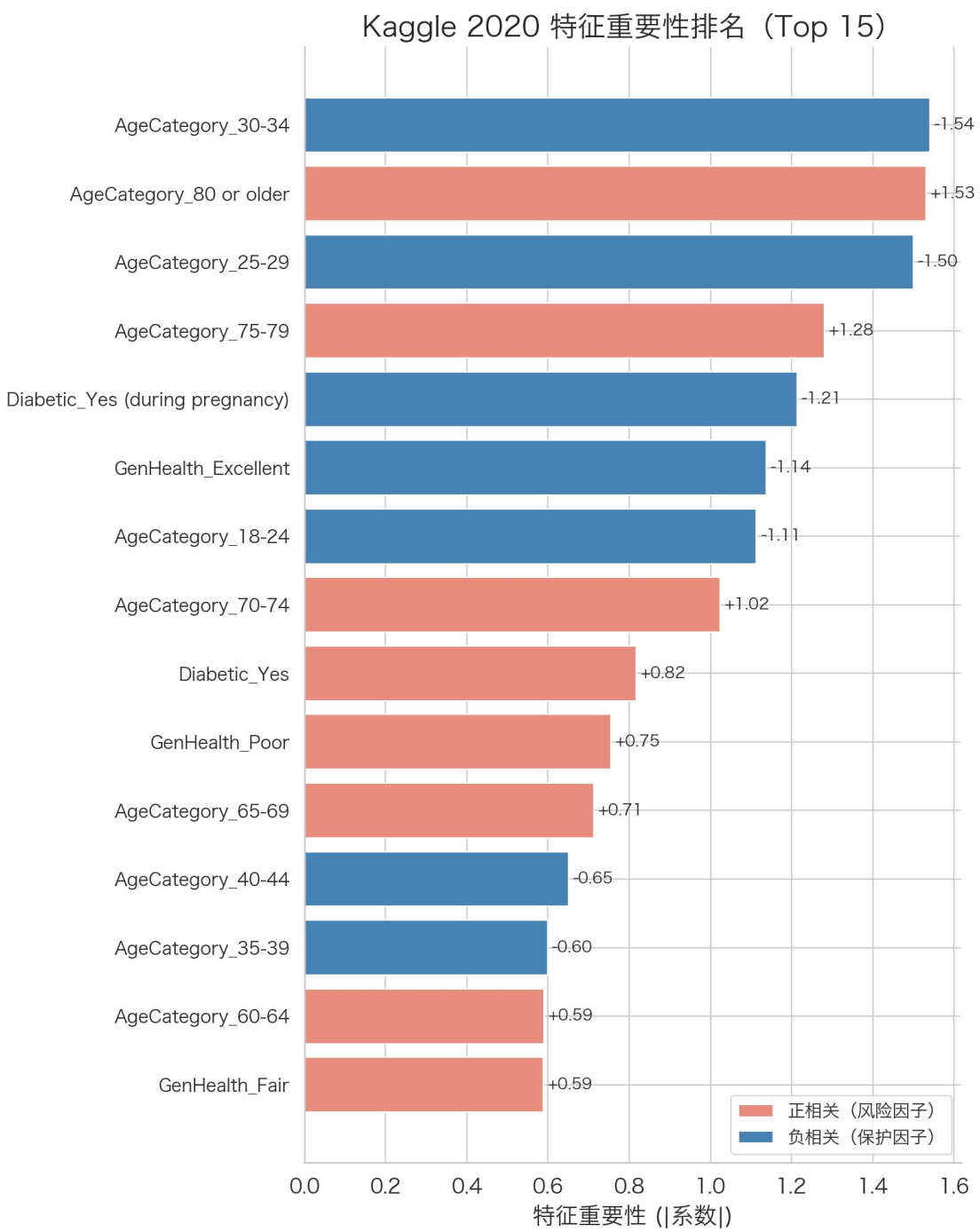

图 4-10 Kaggle 2020 特征重要性排名

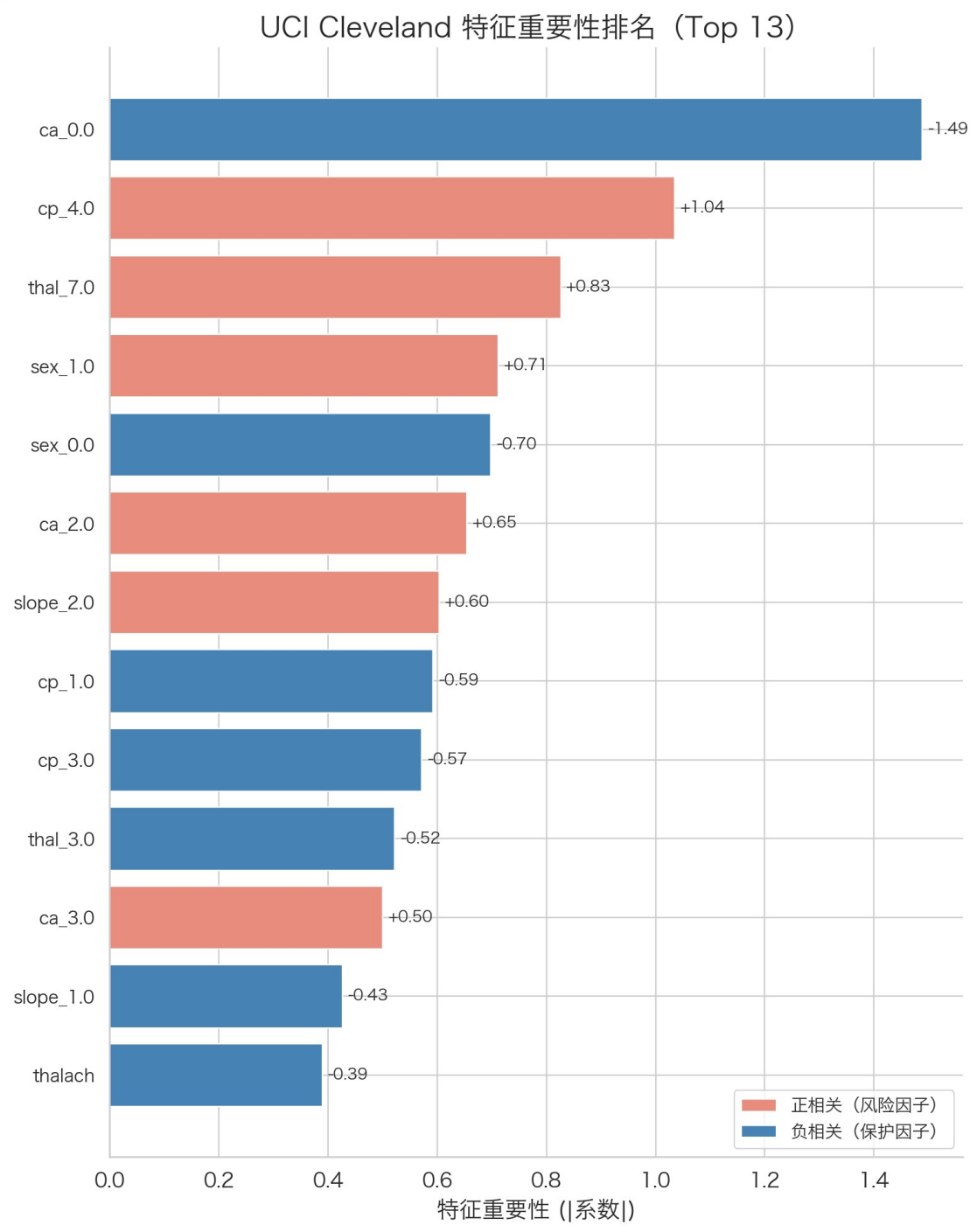

图 4-11 UCI Cleveland 特征重要性排名

综合两个数据集的结果，选择 Kaggle 2020 训练的逻辑回归模型作为系统的在线预测模型——该数据集的输入字段（BMI、年龄段、吸烟等）均为用户可自行填写的健康指标，不需要临床检查。模型的评估指标和特征重要性通过

Python 脚本写入 ADS 层的 ads_model_metrics 和 ads_model_feature_importance 表，同步到 MySQL 后供前端展示。

4.4 本章小结

本章分为两部分。前半部分基于 ADS 聚合表从六个维度展示了心脏病患病率的分布特征，所有数据均来自 Hive 数据仓库的 GROUP BY 聚合结果。后半部分在 DWD 明细数据上训练了四种分类模型，逻辑回归以 AUC=0.821（Kaggle）和 0.967（UCI）胜出。模型指标同样写入 ADS 层，形成了从数据仓库到分析结果的闭环。

5 系统设计与实现

5.1 开发环境

系统开发环境配置如表 5-1 所示。

表 5-1 开发环境配置

类别	工具/版本
操作系统	macOS / Ubuntu 22.04
后端框架	Django 4.2 + DRF 3.15
前端框架	Vue3 + Vite + Naive UI
图表库	ECharts 5
数据库	MySQL 8.0
大数据平台	Hadoop 3.x + Hive 3.x
ML 框架	scikit-learn 1.x
容器化	Docker Compose 2.x
编程语言	Python 3.10 / TypeScript

5.2 需求分析

5.2.1 功能需求

系统功能分成四块：数据分析、机器学习、数据管理、系统管理。下面分别说。

数据分析模块是整个系统里用得最多的部分。ECharts 图表把年龄、性别、BMI、生活习惯、共病史这些维度的患病率统计结果画出来，数据从 MySQL 的 ADS 聚合表里取。用户可以在不同分析页面之间来回切换。

机器学习模块做两件事：一是看模型训练结果——模型对比页面列了四种分类模型的各项指标，特征重要性页面列了哪些因素对心脏病风险影响最大。

二是在线预测——风险预测页面放了一个 16 字段的表单，填完点提交就出结果。

数据管理模块能看到数据仓库的情况：三个数据集各有多少条、清洗了哪些东西、ODS-DWD-ADS 三层表结构长什么样。系统管理模块就是常规的监控、日志、账户管理。

5.2.2 非功能需求

系统非功能需求如表 5-2 所示。

表 5-2 非功能需求说明

需求类别	具体要求
性能	API 响应时间<500ms；预测接口<100ms；页面首次加载<2s
安全性	JWT 令牌认证；密码加密存储；接口权限控制
可用性	界面布局清晰；图表交互流畅；支持主流浏览器
可维护性	前后端分离；模块化代码；Docker 容器化部署

5.3 系统总体设计

系统采用前后端分离的 B/S 架构，总体架构如图 5-1 所示。

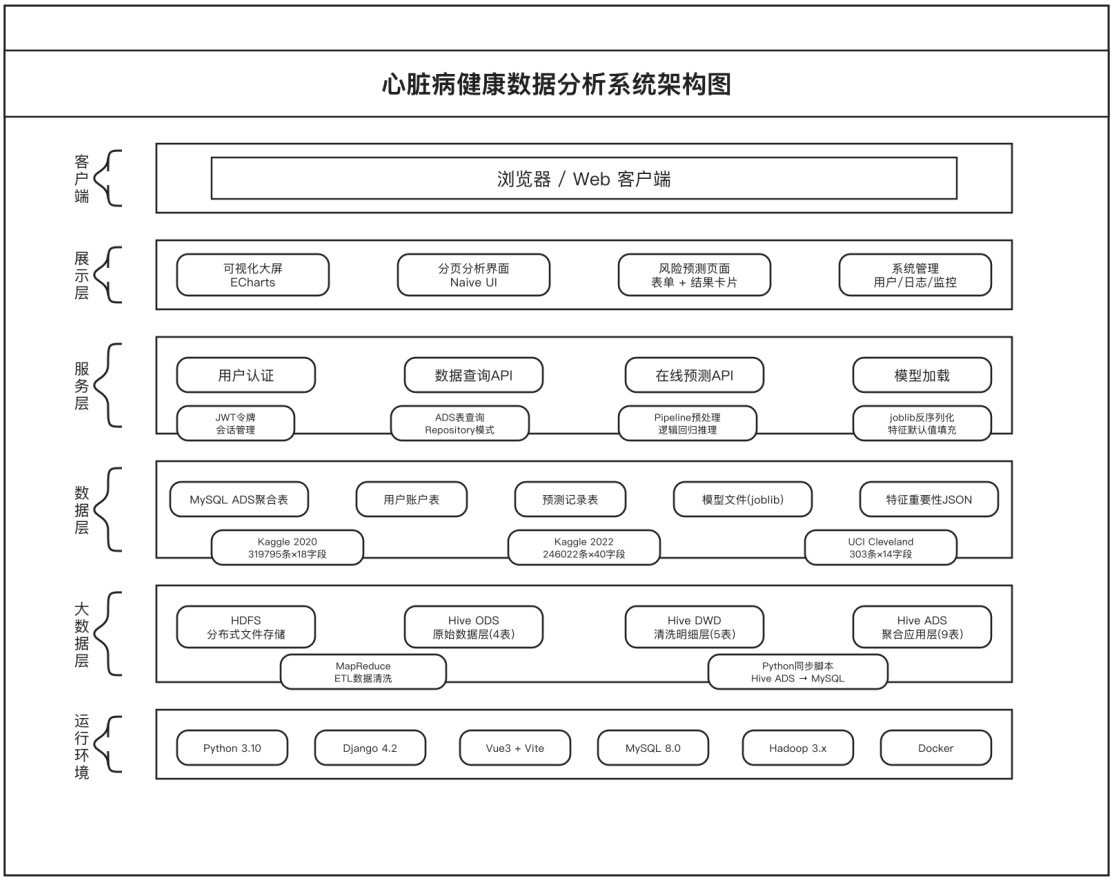

图 5-1 系统总体架构图

数据层由 Hadoop/Hive 数据仓库和 MySQL 两部分组成。原始 CSV 文件存储在 HDFS 上，Hive 负责 ODS-DWD-ADS 三层处理，ADS 层的聚合结果同步到 MySQL。MySQL 同时存储用户账户信息和预测历史记录。服务层为 Django REST Framework 后端，提供 8 个 RESTful API 接口。展示层为 Vue3 单页应用，使用 Naive UI 组件库和 ECharts 图表库。

系统功能模块结构如图 5-2 所示。

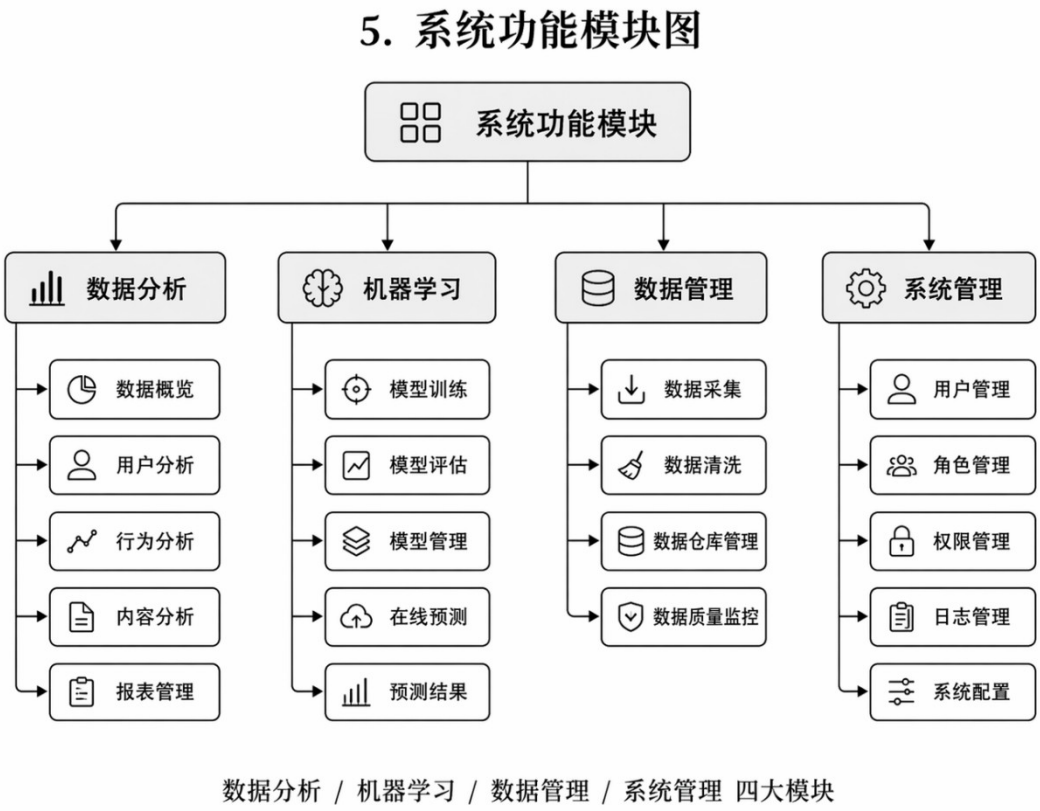

图 5-2 系统功能模块图

在线预测的完整流程如图 5-3 所示。用户在前端填写 16 个健康指标后，系统加载预训练的逻辑回归模型进行推理，根据输出概率划分风险等级并返回结果。

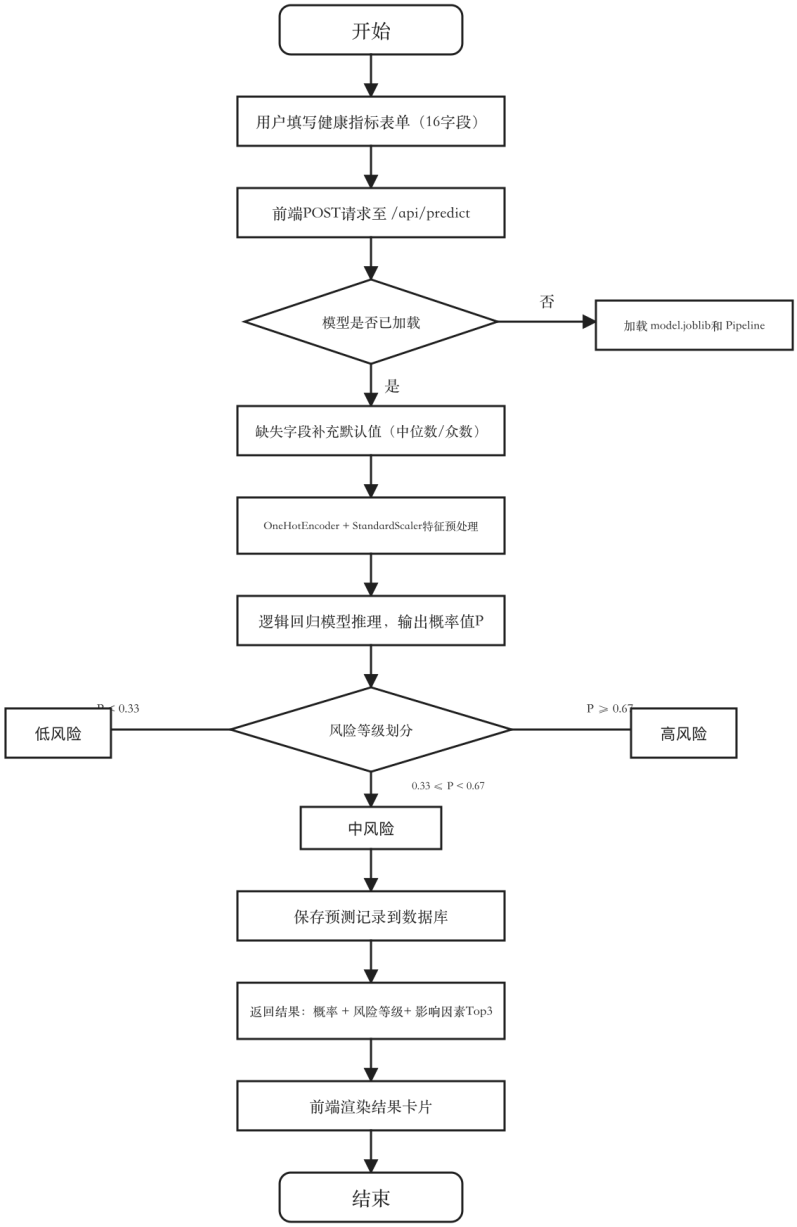

图 5-3 在线预测流程图

表 5-3 列出了主要的 API 接口。

表 5-3 主要 API 接口

接口路径	方法	功能描述
/api/health	GET	服务健康检查
/api/auth/login	POST	用户登录，返回 JWT 令牌
/api/dashboard/overview	GET	总览指标（样本数、患病率）
/api/analysis/age	GET	年龄维度分析数据
/api/analysis/lifestyle	GET	生活习惯和共病维度分析

/api/analysis/clinical	GET	UCI 临床指标分析
/api/model/metrics	GET	模型评估指标和特征重要性
/api/predict	POST	在线心脏病风险预测

预测接口（/api/predict）的输入是 16 个健康指标字段，全部为可选——缺失的字段由模型训练时记录的默认值补全。输出包括风险概率、风险等级（概率≥0.67 为高风险、0.33-0.67 为中风险、0.33 为低风险）和影响当前预测结果最大的三个因素。每次预测结果都写入数据库的 PredictionRecord 表。

5.4 数据库设计

系统使用 MySQL 存储三类数据：ADS 聚合表（9 张，由 Hive 同步而来）、用户账户表和预测记录表。ADS 表的结构已在第三章介绍，此处说明后两张业务表。

用户账户表采用 Django 内置的 auth_user 表，表结构如表 5-4 所示。

表 5-4 用户表（auth_user）结构

字段	类型	说明
id	INT	主键，自增
username	VARCHAR(150)	用户名
password	VARCHAR(128)	加密后的密码
email	VARCHAR(254)	邮箱
is_active	BOOL	账户是否启用
date_joined	DATETIME	注册时间

预测记录表用于保存每次在线预测的输入和输出，表结构如表 5-5 所示。

表 5-5 预测记录表（prediction_record）结构

字段	类型	说明
id	INT	主键，自增
user_id	INT	外键，关联 auth_user
probability	DECIMAL(6,4)	预测概率
risk_level	VARCHAR(16)	风险等级（low/medium/high）
risk_label	VARCHAR(16)	中文标签（低/中/高风险）
model_name	VARCHAR(64)	使用的模型名称
input_data	JSON	用户输入的 16 字段数据
created_at	DATETIME	预测时间

5.5 系统实现

前端有两种看法。一种是大屏模式——全屏铺开，深色背景，一眼能看到患病率、年龄分布、特征重要性、生活方式、UCI 临床指标、数据集对比这些关键数据，见图 5-4。

图 5-4 可视化大屏页面

另一种是分页管理模式，左边栏导航、右边显示内容。左边菜单五个一级模块：可视化大屏、数据分析、机器学习、数据治理、系统运维。数据概览页

面（图 5-5）列了三个数据集的统计信息——数据集总数、总记录数、总存储空间、数据质量评分，饼图画样本量占比，条形图画特征维度对比，下面的表格是每个数据集的细节。

图 5-5 数据概览页面

综合对比分析页面（图 5-6）把三个数据集的样本数、特征数、患病率、阳性样本数摆在一起看。下面还有柱状图和雷达图做横向对比——数据规模差多少、特征丰富度差多少、阳性占比差多少，一张图就看清楚了。

图 5-6 综合对比分析页面

模型性能对比页面（图 5-7）放了 Catboost、逻辑回归、SGD 逻辑回归、随机森林四个模型的结果。上面四张卡片各显示一个模型的 AUC 和四项指标，中间是分组柱状图和雷达图，最下面画了 ROC 曲线，标出来哪个最好。

图 5-7 模型性能对比页面

图 5-8 是 ROC 曲线的放大版。Catboost 的 AUC 是 0.840，比逻辑回归（0.837）和 SGD（0.837）高一点点，随机森林 0.801。四条曲线走势差不多，假阳性率 0.18 的时候 Catboost 的真阳性率到了 0.721。四个模型都远好于随机猜。

图 5-8 ROC 曲线对比

风险预测页面提供了一个 16 字段的表单，用户填写 BMI、年龄段、性别等指标后点击预测按钮，系统调用预训练模型进行推理，返回风险概率、风险等级和影响当前结果最大的三个因素。

5.6 系统测试

对系统的主要功能做了逐项测试，结果如表 5-6 所示。

表 5-6 功能测试用例及结果

测试项	测试内容	预期结果	实际结果
用户登录	输入正确工号和密码	登录成功，跳转首页	通过
数据总览	进入总览页面	显示三个数据集的样本数和患病率	通过
年龄分析	查看年龄分析图表	柱状图和饼图正确渲染	通过
生活习惯分析	查看各因素患病率	六个因素的分组柱状图正确	通过

临床指标分析	选择 UCI 数据集	展示 10 个临床指标的分组统计	通过
模型指标查询	查看模型对比	四种模型的五项指标正确显示	通过
在线预测	填写 16 字段提交	返回风险概率和等级	通过
预测历史	查看历史记录	按时间倒序展示预测记录	通过
大屏展示	切换到大屏模式	全屏展示，图表自适应	通过
Docker 部署	docker compose up	前后端和 MySQL 正常启动	通过

实测下来，预测接口平均响应不到 100 毫秒。时间主要花在 OneHotEncoder 转换上，模型推理本身几毫秒就完事。页面第一次打开 1 到 2 秒，后面切换页面很快。ECharts 画图大概 200 到 300 毫秒，体验上不卡。

5.7 本章小结

本章描述了系统的开发环境、功能需求和非功能需求，介绍了系统的三层架构设计和功能模块划分，说明了数据库表结构设计，展示了可视化大屏、数据概览、综合对比分析、模型性能对比和 ROC 曲线等核心页面的运行效果，最后通过 10 个测试用例验证了系统的功能完整性。

结论

这篇论文围绕心脏病健康数据，做了四件事：建数据仓库、跑多维统计、训练机器学习模型、开发 Web 系统。数据这块，三份公开数据集走了 ODS-DWD-ADS 三层 Hive 仓库，清洗和标准化之后产出 9 张 ADS 聚合表，分别对应年龄、性别、BMI、生活习惯、共病史、UCI 临床指标六个维度。模型这块，Kaggle 数据上逻辑回归 AUC=0.8207、召回率 78.30%，排第一。UCI 数据上同一模型 AUC 到了 0.9665。系统这块，Vue3+ECharts 负责可视化大屏和分页分析界面，Django 后端负责 RESTful 查询和在线预测，Docker Compose 负责容器化部署。

目前的问题也比较明显。Kaggle 数据集阳性只有 8.6%，正负样本差太多，模型精确率只有 23.31%。数据仓库只能离线批处理，没接实时数据流。预测模型也只输出一个概率值，没有可解释性分析。

后面如果要改，样本不平衡的问题可以试 SMOTE 过采样或者 Focal Loss；实时性的问题可以在 Hive 旁边加一路 Spark Streaming；可解释性的问题可以用 SHAP 值给每个预测结果标出哪些特征影响最大。

参考文献

[1]梁靖涵;许亚杰.基于机器学习算法的心脏病预测诊断模型研究[J].现代信息科技,2022:4.

[2]李楠楠.基于神经网络算法建立心血管疾病预测模型[J].信息系统工程,2022:4.

[3]朱宵彤;庞春颖;朱涵.基于深度学习的心血管疾病预测模型[J].计算机应用,2021:5.

[4]吕婷婷;丁子建;袁亦方;林剑平;王贵锦;张萍.深度学习在心电图自动诊断和预测心血管疾病中的应用[J].中国心血管杂志,2021:4.

[5]左维.心血管疾病危险人群风险预测分级与心血管疾病的相关性研究[J].吉林医学,2021:4.

[6]林在宁;杨文杰;陈修洁.基于 Hadoop 的网站大数据分析系统设计[J].北京印刷学院学报,2022:4.

[7]王金元;王宇;张亚松;林昊;龚致富;李盼;安新艳.基于 Hadoop 的分布式财务异常数据分析系统设计[J].信息技术,2022:6.

[8]卢金龙.基于 Hadoop 生态圈的数据分析平台设计[J].金融电子化,2022:2.

[9]宋柯萱.基于 Hadoop 的大数据分析与优化略论[J].电脑知识与技术,2021:3.

[10]吴静.基于改进随机森林的苹果分类系统设计[J].陇东学院学报,2023:7.

[11]聂云丽,高国忠.基于随机森林的页岩气“甜点”分类方法[J].油气藏评价与开发,2023:15.

[12]刘文昌,魏赟,袁浩轩.基于 SMOTE 和 gcForest 的医疗小样本数据分类研究[J].物联网学报,2023:18.

[13]董佳.融合采样算法的信贷不平衡数据分类研究[J].商情,2023:4(41-44).

[14]陈静雯,张鹏鹏,徐思语.基于机器学习的呼吸道疾病预测可视化系统[J].物联网技术,2023:3.

[15]范梓豪,杨楚雯.基于 Vue 的气象信息可视化大屏的设计与实现[J].气象水文海洋仪器,2023:3.

[16]张业男.基于农业数据的可视化大屏教学案例研究[J].牡丹江教育学院学报,2023:4.

[17]郭尚志;章光裕;唐玉玲.基于机器学习的医疗大数据分析与临床应用[J].电脑知识与技术,2022:2.

[18]刘珺;蔡迎;张向阳;孙玮;杜思铭;王佳.医疗大数据分析技术在临床医学中的应用[J].中华医学图书情报杂志,2021:5.

[19]医疗数据分析中隐私保护方法研究[D].山东科技大学,2020.

致谢

毕设做到这里，最想感谢的是指导老师。选题阶段我对大数据方向的了解仅限于课堂上学过的 Hadoop 和 Hive，具体该怎么结合实际场景做一个完整的系统心里没底。老师建议从公开医疗数据集入手，先把数据仓库跑通，再在上面加分析和预测——这条路线事后看确实比一开始就去搞复杂模型要稳妥得多。后来写 SQL 建表、调 Django 接口、画 ECharts 图表的过程中遇到不少问题，老师每次都能给出具体的排查方向，省去了很多在网上漫无目的搜索的时间。

四年本科课程里学到的东西在这次毕设中全部派上了用场。数据结构课上学的哈希表和排序在写 MapReduce 的时候用到了；数据库课上的范式和索引知识在设计 MySQL 表结构时直接套用；机器学习课上跑过的逻辑回归和随机森林模型，这次在真实数据集上重新训练了一遍，体会和课堂作业完全不同。

同学们在环境配置上帮了不少忙。Hadoop 集群搭建、Docker 网络配置这些坑，几个人一起踩比自己摸索快得多。

家人在生活上的支持让我能够把精力集中在毕设上，不用操心其他事情。在此一并致谢。